

UNIVERSIDADE DE FORTALEZA
VICE REITORIA DE GRADUAÇÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Nome do(s) discente(s) integrantes do grupo

DOCUMENTAÇÃO TÉCNICA
TÍTULO DO PROJETO

FORTALEZA
2026

Nome do(s) discente(s) integrantes do grupo

DOCUMENTAÇÃO TÉCNICA
TÍTULO DO PROJETO

Este documento contém a documentação técnica do Sistema [] desenvolvido na componente curricular N392 - Projeto Aplicado Plataformas Web como requisito para obtenção de nota.

Supervisor: Prof.ª Cynthia Moreira, Dra

FORTALEZA
2026

SUMÁRIO

1. INTRODUÇÃO 5
   1.1. CONTEXTO E JUSTIFICATIVA 5
   Exemplo de Texto: 5
   1.2. OBJETIVOS 5
   1.3. ESCOPO E DELIMITAÇÃO 6
2. ENGENHARIA DE REQUISITOS 8
   2.1. REQUISITOS FUNCIONAIS (RFs) 8
   2.2. REQUISITOS NÃO FUNCIONAIS (RNFs) 8
3. PROJETO E ARQUITETURA DO SOFTWARE 10
   3.1. ARQUITETURA GERAL 10
   3.2. PROJETO DO BANCO DE DADOS 10
   3.3. PROJETO DE API 11
4. TECNOLOGIAS E FERRAMENTAS 13
   4.1. STACK DE TECNOLOGIAS 13
   4.2. FERRAMENTAS DE DESENVOLVIMENTO 13
5. IMPLEMENTAÇÃO E RESULTADOS 15
   5.1. TELAS DO SISTEMA 15
6. AMBIENTE E GUIA DE IMPLANTAÇÃO 16
   6.1. REQUISITOS DO AMBIENTE 16
   6.2. PROCESSO DE IMPLANTAÇÃO 16
   6.3. ACESSO À APLICAÇÃO IMPLANTADA 17
7. CONCLUSÃO 18
   7.1. TRABALHOS FUTUROS 18
   7.2. GRUPO DE TRABALHO 18
   7.3. LIÇÕES APRENDIDAS 18

1. INTRODUÇÃO

1.1. CONTEXTO E JUSTIFICATIVA

Instrução: *Nesta seção, você deve apresentar o "mundo" do seu projeto. Descreva o problema real que sua aplicação busca resolver. Quem são as pessoas ou empresas afetadas por esse problema (público-alvo)? Qual a importância ou a oportunidade de negócio em resolver essa questão? Pense em responder à pergunta: "Por que este projeto precisa existir?"*

Exemplo de Texto:

O setor de saúde ainda enfrenta desafios no agendamento e no gerenciamento de consultas médicas. Algumas clínicas ainda utilizam agendas físicas, planilhas eletrônicas ou dependem exclusivamente de atendimento telefônico para gerenciar seus agendamentos, o que pode ocasionar atrasos, marcações duplicadas, faltas às consultas e conflitos de horários.

Além disso, o tempo da equipe administrativa está sobrecarregado por tarefas operacionais que poderiam ser automatizadas. O projeto "ConsultaFácil" propõe a criação de uma plataforma digital simples e acessível que permita aos pacientes agendar, reagendar ou cancelar consultas diretamente pelo celular, além de enviar lembretes automáticos e integrar a agenda dos médicos em tempo real. O objetivo é reduzir o índice de faltas, melhorar o aproveitamento da agenda médica e oferecer uma experiência mais eficiente tanto para o paciente quanto para a clínica. A necessidade de modernização nesse nicho representa uma oportunidade real de negócio, especialmente em regiões urbanas com alta demanda por serviços de saúde acessíveis e bem organizados.

1.2. OBJETIVOS

Instrução: *Aqui você deve listar, de forma clara e direta, o que o projeto pretende alcançar. O* ***Objetivo Geral*** *é a grande missão do sistema. Os* ***Objetivos Específicos*** *são as metas menores e mensuráveis que, somadas, alcançam o objetivo geral.*

Exemplo de Texto

O objetivo deste projeto é desenvolver um sistema web e mobile para facilitar e automatizar o agendamento de consultas médicas, promovendo maior organização, maior eficiência no atendimento e redução de ausências.

Dito isso, os objetivos específicos do projeto são:

- *Implementar um módulo de agendamento online que permita ao paciente marcar, remarcar ou cancelar consultas de forma simples e rápida.*
- *Criar uma agenda digital integrada para os profissionais de saúde e a equipe administrativa, com visualização em tempo real.*
- *Desenvolver um sistema de notificações automáticas por e-mail e/ou WhatsApp para lembrar os pacientes das consultas agendadas.*
- *Disponibilizar um painel de gestão para controle da ocupação de horários, número de atendimentos e estatísticas de faltas.*

1.3. ESCOPO E DELIMITAÇÃO

Instrução: *Seja extremamente nítido sobre o que o seu projeto* ***FAZ*** *(Escopo) e o que ele* ***NÃO FAZ*** *(Delimitação). Isso é crucial para gerenciar as expectativas da banca avaliadora e demonstrar maturidade no planejamento.*

Exemplo de Texto:

- ***Escopo:***
    - *Cadastro, edição, visualização e exclusão de pacientes e profissionais de saúde.*
    - *Agendamento, remarcação e cancelamento de consultas médicas por meio de interface web/mobile.*
    - *Visualização de agenda em tempo real, com bloqueios e disponibilidades configuráveis pelos administradores.*

- *Envio de notificações automáticas (por e-mail e/ou WhatsApp) para lembrar os pacientes de suas consultas.*
- *Controle de acesso com diferentes perfis de usuário (administrador, recepcionista, médico, paciente).*
  Geração de relatórios básicos sobre agendamentos realizados, consultas perdidas (no-show) e ocupação da agenda.
-

- ***Delimitação (Fora do Escopo):***
  - O sistema não incluirá funcionalidades de prontuário eletrônico ou registro clínico dos pacientes.
  - Não haverá integração com sistemas de operadoras de planos de saúde ou convênios.
  - O sistema não realizará cobrança ou gestão financeira (como pagamentos, boletos, ou emissão de notas fiscais).
  - Não será implementado suporte para telemedicina ou videochamadas dentro da plataforma.
  - O sistema não contemplará fila de espera automatizada ou algoritmos avançados de alocação dinâmica de horários.

2. ENGENHARIA DE REQUISITOS

2.1. REQUISITOS FUNCIONAIS (RFs)

Instrução: *Liste as ações e funcionalidades que o sistema deve ser capaz de executar. Use uma tabela para organizar, com um ID único, um nome e uma breve descrição para cada requisito.*

ID	Nome do Requisito	Descrição
RF01	Cadastrar Paciente	O sistema deve permitir o cadastro de pacientes, incluindo nome completo, CPF, data de nascimento, telefone e e-mail.
RF02	Cadastrar Profissional de Saúde	O sistema deve permitir o cadastro de médicos ou outros profissionais, com informações como nome, especialidade, CRM e horários de atendimento.
RF03	Agendar Consulta	O sistema deve permitir que pacientes ou recepcionistas agendem consultas, selecionando data, hora e profissional disponível.
RF04	Cancelar ou Remarcar Consulta	O sistema deve permitir o cancelamento ou remarcação de consultas já agendadas, respeitando regras de horário mínimo para alteração.
RF05	Autenticar Usuário	O sistema deve permitir que usuários (pacientes, médicos, recepcionistas e administradores) façam login por meio de e-mail e senha.

2.2. REQUISITOS NÃO FUNCIONAIS (RNFs)

Instrução: *Liste as características de qualidade do sistema. Eles não são "o que" o sistema faz, mas "como" ele faz. Separe por categorias para facilitar a compreensão.*

Exemplo de Texto:

- **Usabilidade:**
  - RNF01: A interface deve ser intuitiva, permitindo que um paciente consiga agendar uma consulta sem necessidade de treinamento, em menos de 5 minutos de navegação.
- **Desempenho:**
  - RNF02: A página de agendamento deve carregar em até 2 segundos, mesmo com dezenas de horários e profissionais disponíveis.
  - RNF03: A resposta da API para consulta de horários disponíveis deve ocorrer em no máximo 300ms.
- **Segurança:**
  - RNF04: Toda a comunicação entre o frontend e o backend deve ocorrer via protocolo HTTPS.
  - RNF05: As senhas dos usuários devem ser armazenadas com algoritmo de hashing seguro (como bcrypt), incluindo uso de salt único por senha.
  - RNF06**:** O sistema deve respeitar os princípios da LGPD, garantindo o consentimento e a proteção dos dados pessoais dos pacientes.
- **Compatibilidade:**
  - RNF06: A aplicação deve ser compatível com as duas últimas versões dos navegadores Google Chrome, Mozilla Firefox, Microsoft Edge e Safari.

3. PROJETO E ARQUITETURA DO SOFTWARE

3.1. ARQUITETURA GERAL

Instrução: *Apresente um diagrama de alto nível da arquitetura do seu sistema. Pode ser um diagrama de camadas, de componentes, ou outro que represente bem a sua solução. Justifique brevemente por que você escolheu esse padrão arquitetural.*

Exemplo de Texto: O sistema **ConsultaFácil** foi projetado utilizando uma **Arquitetura em 3 Camadas**, garantindo organização, manutenibilidade e escalabilidade. As camadas são:

1. **Camada de Apresentação (Frontend):** Desenvolvida como uma **Single Page Application (SPA)** utilizando o framework **React**. Essa camada é responsável por toda a interface com o usuário, tanto para pacientes quanto para profissionais de saúde e administradores. A escolha do React se deu por sua capacidade de criar interfaces dinâmicas, responsivas e reutilizáveis, além de contar com uma ampla comunidade e integração com bibliotecas modernas como React Router e Axios.
2. **Camada de Aplicação (Backend API):** Uma API RESTful desenvolvida em Node.js com o framework Express.js. Ela contém toda a lógica de negócio, processa as requisições do frontend e se comunica com o banco de dados. Node.js foi escolhido por sua natureza não-bloqueante, ideal para operações de I/O intensivas como consultas ao banco e manipulação de arquivos de imagem.
3. **Camada de Dados (Banco de Dados):** Um banco de dados relacional PostgreSQL, responsável pela persistência e integridade dos dados. O PostgreSQL foi escolhido por sua robustez, suporte a tipos de dados complexos e confiabilidade transacional.

[INSERIR IMAGEM DO DIAGRAMA DE ARQUITETURA AQUI]

3.2. PROJETO DO BANCO DE DADOS

Instrução: *Insira o Diagrama Entidade-Relacionamento (DER) do seu banco de dados. Logo após, crie um Dicionário de Dados para as tabelas mais importantes, detalhando cada campo.*

Exemplo de Texto:

Diagrama Entidade-Relacionamento (DER): **[INSERIR IMAGEM DO DER AQUI]**

Dicionário de Dados (Tabela **Consultas**):

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	UUID	PK	Não	Identificador único universal da consulta
profissional_id	UUID	FK		Chave estrangeira para a tabela Profissionais
motivo_cancelamento	VARCHAR(255)			Justificativa para cancelamento (se aplicável)
status	VARCHAR(20)			Status da consulta: "Agendada", "Cancelada", "Realizada", "Faltou"
paciente_id	UUID	FK		Chave estrangeira para a tabela Pacientes
data_hora	TIMESTAMPZ			Data e hora agendada para a consulta

3.3. PROJETO DE API

Instrução: *Em vez de documentar a API em uma tabela estática, a prática moderna é usar a Especificação OpenAPI para criar uma documentação interativa e centralizada. Nesta seção, você deve explicar que a API do projeto segue esse padrão e fornecer o link para a documentação "viva", onde qualquer desenvolvedor (incluindo a equipe mobile ou a banca avaliadora) pode explorar e testar cada endpoint. A equipe de backend é responsável por criar o arquivo* `openapi.yaml` *e usar uma ferramenta como o Swagger UI para gerar a página de documentação. Este link é a "fonte da verdade" da sua API.*

Exemplo de Texto:

A API do sistema **ConsultaFácil** é documentada seguindo a **Especificação OpenAPI 3.0**, que funciona como um contrato formal e uma fonte única de verdade para todos os seus endpoints.

Esta abordagem permite a geração de uma documentação interativa, onde cada endpoint pode ser inspecionado e testado em tempo real, facilitando a integração com o frontend e com outras aplicações, como o aplicativo mobile.

Link para a Documentação Interativa da API:

- **URL:** *(substitua pelo link real da sua documentação Swagger UI/Redoc)*

A seguir, um exemplo visual da nossa documentação interativa, mostrando o detalhe do endpoint para cadastrar uma nova consulta médica:

Figura 4: Interface da Documentação da API com Swagger UI *(Legenda: Detalhes do endpoint `POST /consultas`, mostrando os parâmetros esperados, o schema do corpo da requisição e as possíveis respostas.)*

`[INSERIR SCREENSHOT DA INTERFACE DO SWAGGER UI MOSTRANDO UM ENDPOINT EXPANDIDO]`

Para fins de registro e consulta offline, o conteúdo completo do arquivo `openapi.yaml` que gera esta documentação está disponível no Apêndice A deste documento (ou no repositório do projeto).

4. TECNOLOGIAS E FERRAMENTAS

4.1. STACK DE TECNOLOGIAS

Instrução: *Liste aqui todas as linguagens, frameworks e bibliotecas principais que foram utilizadas para construir o projeto. É importante não apenas listar, mas também justificar brevemente as escolhas mais significativas. Por que essa tecnologia foi a melhor opção para o seu projeto?*

Exemplo de Texto:

- ***Frontend:***
  - ***React 18:*** *Escolhido para a construção da interface do usuário por sua arquitetura baseada em componentes, que facilita a reutilização de código e a manutenção. O ecossistema robusto e a vasta comunidade foram decisivos para a escolha.*
  - ***Vite:*** *Utilizado como ferramenta de build por sua incrível velocidade de desenvolvimento (Hot Module Replacement) e configuração simplificada.*
  - ***Axios:*** *Biblioteca para realizar as requisições HTTP à nossa API backend, escolhida por sua simplicidade e facilidade de uso.*
  - ***Tailwind CSS:*** *Framework de CSS utility-first que nos permitiu construir interfaces customizadas de forma rápida e consistente sem sair do HTML.*
- ***Backend:***
  - ***Node.js v20:*** *Ambiente de execução escolhido por sua eficiência em operações de entrada e saída (I/O), ideal para uma API que lida com muitas requisições de rede e consultas ao banco de dados.*
  - ***Express.js 4:*** *Framework minimalista para Node.js, utilizado para estruturar nossa API RESTful, definindo rotas e middlewares.*
  - ***Prisma ORM:*** *Escolhido para fazer a comunicação com o banco de dados. Sua tipagem automática e a facilidade para escrever consultas complexas aumentaram nossa produtividade e a segurança do código.*
  - ***JSON Web Tokens (JWT):*** *Utilizado para implementar a autenticação e autorização, garantindo que apenas usuários autenticados possam acessar as rotas protegidas da API.*

4.2. FERRAMENTAS DE DESENVOLVIMENTO

Instrução: *Liste as ferramentas que apoiaram o processo de desenvolvimento, desde a escrita do código até o gerenciamento das tarefas da equipe.*

Exemplo de Texto:

- ***IDE:*** *Visual Studio Code foi a IDE padrão para toda a equipe, devido à sua leveza, extensibilidade e terminal integrado.*
- ***Controle de Versão:*** *Git, com o repositório hospedado no GitHub. Adotamos o fluxo de trabalho "GitFlow", com branches separadas para* `develop`*,* `features` *e* `main`*.*
- ***Gerenciamento de Projeto:*** *Trello foi utilizado para gerenciar as tarefas. Criamos um quadro Kanban com as colunas "A Fazer", "Em Andamento", "Em Teste" e "Concluído".*
- ***Ferramenta de API:*** *Insomnia foi usado para testar os endpoints da API durante o desenvolvimento.*

5. IMPLEMENTAÇÃO E RESULTADOS

5.1. TELAS DO SISTEMA

Instrução: *Esta é a vitrine do seu projeto. Insira imagens (screenshots) das principais telas da sua aplicação. Cada imagem deve ter uma legenda curta e clara explicando sua finalidade. Dê preferência a telas que demonstrem as funcionalidades mais importantes (RFs) que você listou na seção 2.*

Exemplo de Texto:

Figura 1: Tela de Login *(Legenda: A tela de login é o ponto de entrada do sistema, garantindo o acesso seguro através de autenticação por e-mail e senha.)*

[INSERIR SCREENSHOT DA TELA DE LOGIN AQUI]

Figura 2: Dashboard Principal do Profissional de Saúde *(Legenda: Após o login, o profissional visualiza um dashboard com sua agenda diária, consultas agendadas, horários disponíveis e notificações importantes.)*

[INSERIR SCREENSHOT DO DASHBOARD AQUI]

Figura 3: Tela de Agendamento de Consulta *(Legenda: Formulário intuitivo para o agendamento de consultas, onde o paciente ou recepcionista seleciona o profissional, data, hora e pode adicionar observações.)*

[INSERIR SCREENSHOT DO FORMULÁRIO DE CADASTRO AQUI]

6. AMBIENTE E GUIA DE IMPLANTAÇÃO

Instrução: *Esta seção é um mini manual de operações. Imagine que um novo desenvolvedor precise instalar e rodar o seu projeto do zero. As instruções devem ser claras e completas o suficiente para que ele consiga fazer isso sem ajuda. Seja específico com as versões dos softwares.*

6.1. REQUISITOS DO AMBIENTE

Instrução: *Liste todo o software necessário para rodar o projeto localmente, incluindo o sistema operacional, linguagens, banco de dados, etc.*

Exemplo de Texto:

- **Sistema Operacional:** Ubuntu 22.04 LTS (ou superior) / Windows 10 com WSL2
- **Runtime:** Node.js v20.11.0
- **Gerenciador de Pacotes:** npm v10.2.4
- **Banco de Dados:** PostgreSQL v15
- **Git:** v2.34.1

6.2. PROCESSO DE IMPLANTAÇÃO

Instrução: *Forneça uma sequência de comandos que devem ser executados no terminal para configurar e iniciar o projeto. Inclua os passos para o backend e para o frontend.*

Exemplo de Texto:

Backend (API):

1. *Clone o repositório:* `git clone https://github.com/equipe/consulta-api.git`
2. *Entre na pasta do projeto:* `cd consulta-api`
3. *Instale as dependências:* `npm install`
4. *Copie o arquivo de variáveis de ambiente:* `cp .env.example .env`

5. ***Importante:*** *Edite o arquivo* `.env` *e configure a* `DATABASE_URL` *com a sua string de conexão do PostgreSQL.*
6. *Execute as migrações do banco de dados:* `npx prisma migrate deploy`
7. *Inicie o servidor da API (ele rodará na porta 3333):* `npm run dev`

Frontend (Aplicação Web):

1. *Em um novo terminal, clone o repositório:* `git clone https://github.com/equipe/consulta-web.git`
2. *Entre na pasta do projeto:* `cd consulta-web`
3. *Instale as dependências:* `npm install`
4. *Inicie a aplicação de desenvolvimento (ela rodará na porta 5173):* `npm run dev`
5. *Abra o navegador e acesse* `http://localhost:5173`.

6.3. ACESSO À APLICAÇÃO IMPLANTADA

Instrução: *Se o seu projeto estiver hospedado em algum lugar na internet (Vercel, Heroku, etc.), coloque o link aqui. Forneça também um usuário de teste para que a banca possa navegar pelo sistema.*

Exemplo de Texto:

- ***URL da Aplicação:*** `https://consulta.vercel.app`
- ***Credenciais de Acesso (Perfil de Vendedor):***
  - ***Usuário:*** `vendedor@teste.com`
  - ***Senha:*** `vendedor123`

7. CONCLUSÃO

7.1. TRABALHOS FUTUROS

Instrução: *Nenhum projeto está 100% completo. Liste aqui as melhorias e novas funcionalidades que você gostaria de implementar no futuro. Isso demonstra visão de produto e consciência das limitações do trabalho atual.*

Exemplo de Texto:

- Implementação de um módulo de relatórios avançados com gráficos interativos.
- Criação de um portal para o cliente final acompanhar o status de seus pedidos.
- Desenvolvimento de um aplicativo mobile para os vendedores usarem no pátio.
- Integração com um sistema de gestão de entregas (logística).

7.2. GRUPO DE TRABALHO

Apresentar as funções, responsabilidades e atividades específicas de cada integrante do grupo de trabalho.

7.3. LIÇÕES APRENDIDAS

Instrução: *Faça uma breve reflexão sobre a jornada do projeto. Quais foram os maiores desafios técnicos? E os desafios de trabalhar em equipe? O que vocês aprenderam no processo que levarão para os próximos projetos?*

Exemplo de Texto: O desenvolvimento do sistema ConsultaFácil foi uma experiência muito enriquecedora, tanto do ponto de vista técnico quanto interpessoal. O maior desafio técnico enfrentado foi garantir a sincronização e a consistência dos agendamentos em tempo real, evitando conflitos de horários e problemas de concorrência quando múltiplos usuários tentavam marcar consultas simultaneamente. Também foi desafiador implementar um sistema de notificações eficiente que funcionasse de forma confiável em diferentes canais, como e-mail e WhatsApp.No aspecto de trabalho em equipe, aprendemos a importância da comunicação constante e clara, especialmente em um projeto remoto. Dividir as tarefas em etapas menores e bem definidas ajudou a manter o foco e o progresso constante. A adoção de reuniões rápidas e periódicas para alinhar o andamento e resolver dúvidas contribuiu significativamente para o sucesso do projeto.

A principal lição que levaremos para projetos futuros é que um planejamento cuidadoso da arquitetura do sistema e da divisão de responsabilidades evita muitos retrabalhos e facilita a integração entre as partes. Além disso, a colaboração efetiva da equipe, aliada a ferramentas ágeis de gestão e comunicação, é essencial para o cumprimento dos prazos e a qualidade do produto final.